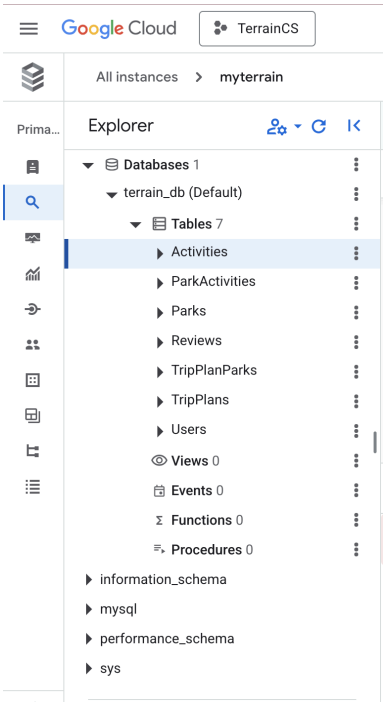
Database Connection



DDL Commands for tables

```sql
CREATE DATABASE terrain_db;
USE terrain_db;

CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at DATETIME,
    updated_at DATETIME
);

CREATE TABLE Parks (
    park_id INT PRIMARY KEY AUTO_INCREMENT,
    park_code VARCHAR(10) UNIQUE NOT NULL,
    park_name VARCHAR(100) NOT NULL,
    description TEXT,
    state VARCHAR(50),
    latitude DECIMAL(10,8),
    longitude DECIMAL(11,8),
    operating_hours TEXT,
    entrance_fees TEXT,
    contact_info TEXT
);
```

```sql
CREATE TABLE Activities (
    activity_id INT PRIMARY KEY AUTO_INCREMENT,
    activity_name VARCHAR(100) NOT NULL,
    description TEXT
);


CREATE TABLE TripPlans (
    trip_plan_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    plan_name VARCHAR(100),
    created_at DATETIME,
    updated_at DATETIME,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);


CREATE TABLE TripPlanParks (
    trip_plan_id INT NOT NULL,
    park_id INT NOT NULL,
    added_at DATETIME,
    user_notes TEXT,
    PRIMARY KEY (trip_plan_id, park_id),
    FOREIGN KEY (trip_plan_id) REFERENCES TripPlans(trip_plan_id),
    FOREIGN KEY (park_id) REFERENCES Parks(park_id)
);


CREATE TABLE Reviews (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    park_id INT NOT NULL,
    rating INT,
    review_text TEXT,
    created_at DATETIME,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (park_id) REFERENCES Parks(park_id)
);

CREATE TABLE ParkActivities (
    park_id INT NOT NULL,
    activity_id INT NOT NULL,
    PRIMARY KEY (park_id, activity_id),
    FOREIGN KEY (park_id) REFERENCES Parks(park_id),
    FOREIGN KEY (activity_id) REFERENCES Activities(activity_id)
);
```

Inserting at least 1000 entries:

Run Save Format Clear Valid

```sql
SELECT
  (SELECT COUNT(*) FROM `TripPlans`) AS TripPlanCount,
  (SELECT COUNT(*) FROM `Reviews`) AS ReviewCount,
  (SELECT COUNT(*) FROM `Users`) AS UserCount;
```

Results Execution time: 1.0 ms Export

TripPlanCount	ReviewCount	UserCount
1000	1016	2000

Rows per page: 100 1 – 1 of 1

Use a script or loop to generate 1,000 users:

Run Save Format Clear

```sql
CREATE PROCEDURE `Insert1000Users`()
BEGIN
    -- Declare a counter variable and initialize it to 1
    DECLARE i INT DEFAULT 1;

    -- The WHILE loop runs as long as i is less than or equal to 1000
    WHILE i <= 1000 DO
        -- Generate and insert a unique user record
        INSERT INTO `Users` (`username`, `email`, `password_hash`, `created_at`, `updated_at`)
        VALUES (
            CONCAT('adventurer_', LPAD(i, 4, '0')), -- e.g., adventurer_0001
            CONCAT('email', i, '@terrainapp.com'),
            -- Using a strong, consistent hash for the password
            SHA2(CONCAT('a_secure_password_', i), 256),
            NOW(),
            NOW()
        );

        -- Increment the counter
        SET i = i + 1;
    END WHILE;
END;

CALL `Insert1000Users`();

DROP PROCEDURE IF EXISTS `Insert1000Users`;
```

Results

Statement executed successfully

```sql
INSERT INTO `Reviews` (`user_id`, `park_id`, `rating`, `review_text`, `created_at`)
SELECT
    (SELECT `user_id` FROM `Users` ORDER BY RAND() LIMIT 1) AS `random_user_id`,
    (SELECT `park_id` FROM `Parks` ORDER BY RAND() LIMIT 1) AS `random_park_id`,
    TRUNCATE(1 + (RAND() * 5), 0) AS `random_rating`,
    CONCAT('Final Batch Review #', numbers.n + 984),
    NOW()
FROM
    (
        SELECT 1 AS n UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4 UNION ALL
        SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8 UNION ALL
        SELECT 9 UNION ALL SELECT 10 UNION ALL SELECT 11 UNION ALL SELECT 12 UNION ALL
        SELECT 13 UNION ALL SELECT 14 UNION ALL SELECT 15 UNION ALL SELECT 16
    ) AS numbers;

SELECT COUNT(*) FROM `Reviews`;
```

Results

COUNT(*)
1016

This code uses a CROSS JOIN technique on an auxiliary table to insert reviews and now there are over 1000 reviews

4 Advanced Queries:

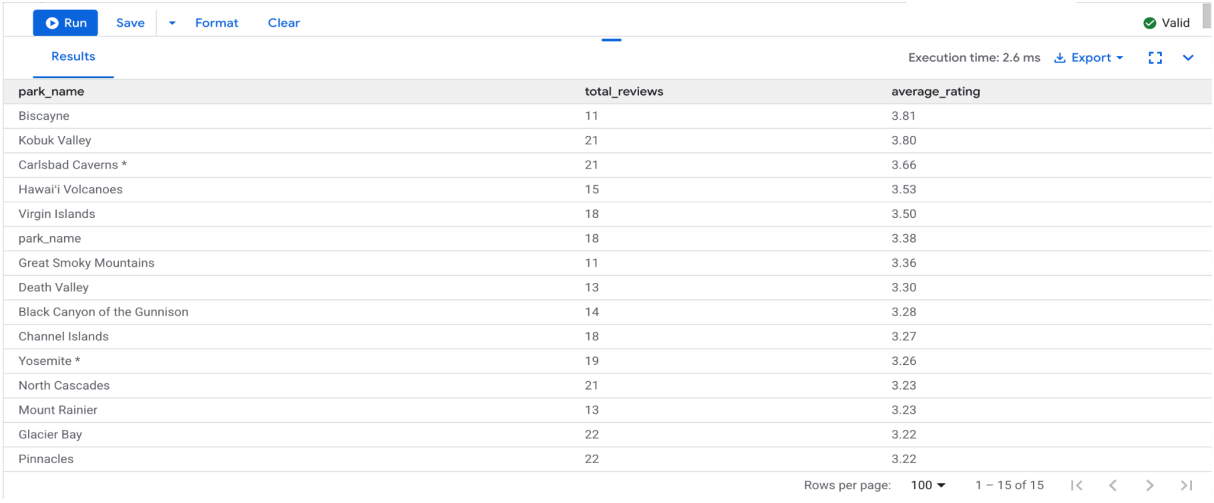
Query 1: Top-Rated Parks by Average Rating and Review Count

This query identifies the parks with an **above-average overall rating** that also have **more than a specific minimum number of reviews** (e.g., 5 reviews) to ensure statistical relevance.

Concepts Used: Aggregation via GROUP BY, Join multiple relations, Subquery (in HAVING).

Run Save Format Clear

```sql
SELECT
    p.park_name,
    COUNT(r.review_id) AS total_reviews,
    TRUNCATE(AVG(r.rating), 2) AS average_rating
FROM
    Parks p
JOIN
    Reviews r ON p.park_id = r.park_id
GROUP BY
    p.park_id, p.park_name
HAVING
    -- Filter for parks with a rating higher than the overall average rating
    AVG(r.rating) > (SELECT AVG(rating) FROM Reviews)
    -- And at least 5 reviews for relevance
    AND COUNT(r.review_id) >= 5
ORDER BY
    average_rating DESC, total_reviews DESC
LIMIT 15;
```



park_name	total_reviews	average_rating
Biscayne	11	3.81
Kobuk Valley	21	3.80
Carlsbad Caverns *	21	3.66
Hawai'i Volcanoes	15	3.53
Virgin Islands	18	3.50
park_name	18	3.38
Great Smoky Mountains	11	3.36
Death Valley	13	3.30
Black Canyon of the Gunnison	14	3.28
Channel Islands	18	3.27
Yosemite *	19	3.26
North Cascades	21	3.23
Mount Rainier	13	3.23
Glacier Bay	22	3.22
Pinnacles	22	3.22

Query 2: Most Active Trip Planners and Reviewers

This query helps identify the **15 most engaged users** by combining **counts of their trip plans and reviews** and ranking them. This is valuable for community features and user-engagement metrics.

Concepts Used: Aggregation via GROUP BY, Join multiple relations, SET Operators (UNION ALL), Common Table Expression (CTE).

```sql
WITH UserActivityCounts AS (
    -- Count of Trip Plans per user
    SELECT
        user_id,
        COUNT(trip_plan_id) AS total_count,
        'Trip Plans' AS activity_type
    FROM
        TripPlans
    GROUP BY
        user_id

    UNION ALL -- SET Operator: Combines results, includes duplicates if a user has both activities

    -- Count of Reviews per user
    SELECT
        user_id,
        COUNT(review_id) AS total_count,
        'Reviews' AS activity_type
    FROM
        Reviews
    GROUP BY
        user_id
)

SELECT
    u.username,
    SUM(uac.total_count) AS combined_activity_count -- Aggregation
FROM
    UserActivityCounts uac
JOIN
    Users u ON uac.user_id = u.user_id
GROUP BY
    u.username
ORDER BY
    combined_activity_count DESC
LIMIT 15;
```

Results

Execution time: 3.7 ms Export

username	combined_activity_count
lopeztammy	8
ryan92	7
ldonaldson	7
lwilliams	7
ramosmichael	7
williamcherry	7
kristen57	6
joshuathompson	6
wellsbrian	6
jamesdavid	6
llawrence	6
kfaulkner	6
qrasmussen	6
yholloway	6
jennifervargas	6

Rows per page: 100 1 – 15 of 15

Query 3: Parks with Above-Average User Notes for a Trip Plan

This query finds parks that users tend to spend more time detailing in their trip plans, **based on the average user notes length**, suggesting they are **complex or highly anticipated destinations**. This

leverages the user_notes attribute in the TripPlanParks junction table. Find parks whose average user notes length in trip plans is longer than the average note length across all parks

Concepts Used: Aggregation via GROUP BY, Join multiple relations, Subquery (in HAVING).

```sql
SELECT
    p.park_name,
    p.state,
    AVG(LENGTH(tpp.user_notes)) AS average_note_length -- Aggregation
FROM
    Parks p
JOIN
    TripPlanParks tpp ON p.park_id = tpp.park_id
GROUP BY
    p.park_name, p.state
HAVING
    AVG(LENGTH(tpp.user_notes)) > (
        -- Subquery: Calculate the overall average length of all user notes
        SELECT
            AVG(LENGTH(user_notes))
        FROM
            TripPlanParks
        WHERE
            user_notes IS NOT NULL
    )
ORDER BY
    average_note_length DESC
```

Results

Execution time: 1.0 ms Export

park_name	state	average_note_length
Hawai'i Volcanoes	Hawaii19°23'N 155°12'W / 19.38°N 155.20°W	85.0000
Joshua Tree	California33°47'N 115°54'W / 33.79°N 115.90°W	69.0000
Big Bend	Texas29°15'N 103°15'W / 29.25°N 103.25°W	68.0000
Kobuk Valley	Alaska67°33'N 159°17'W / 67.55°N 159.28°W	65.0000
Black Canyon of the Gunnison	Colorado38°34'N 107°43'W / 38.57°N 107.72°W	65.0000
Saguaro	Arizona32°15'N 110°30'W / 32.25°N 110.50°W	63.0000
Wrangell–St. Elias *	Alaska61°00'N 142°00'W / 61.00°N 142.00°W	62.3333
Indiana Dunes	Indiana41°39'12"N 87°03'09"W / 41.6533°N 87.0524	60.0000
Great Smoky Mountains	North Carolina, Tennessee35°41'N 83°32'W / 35.68	59.0000
Theodore Roosevelt	North Dakota46°58'N 103°27'W / 46.97°N 103.45°W	58.0000
Yellowstone	Wyoming, Montana, Idaho44°36'N 110°30'W / 44.60°	57.0000
New River Gorge	West Virginia38°04'N 81°05'W / 38.07°N 81.08°W	57.0000
Guadalupe Mountains	Texas31°55'N 104°52'W / 31.92°N 104.87°W	56.0000
Mount Rainier	Washington46°51'N 121°45'W / 46.85°N 121.75°W	55.0000
Hot Springs	Arkansas34°31'N 93°03'W / 34.51°N 93.05°W	55.0000

Query 4: Park Ratings vs. User's Rating Trend

Identify parks that a user gave a rating to that is significantly lower than their **personal average rating**. This could flag potential negative outliers or indicate a genuinely disappointing park experience for that user.

Concepts Used: Subquery (to calculate user's average rating), Join multiple relations, Conditional filtering

Run Save Format Clear

```sql
SELECT
    u.username,
    p.park_name,
    r.rating AS actual_rating,
    user_avg.avg_user_rating
FROM
    Users u
JOIN
    Reviews r ON u.user_id = r.user_id
JOIN
    Parks p ON r.park_id = p.park_id
JOIN
    (
        SELECT user_id, AVG(rating) AS avg_user_rating
        FROM Reviews
        GROUP BY user_id
    ) AS user_avg ON u.user_id = user_avg.user_id
WHERE
    r.rating < (user_avg.avg_user_rating - 1.0)
ORDER BY
    (user_avg.avg_user_rating - r.rating) DESC
LIMIT 15;
```

Results

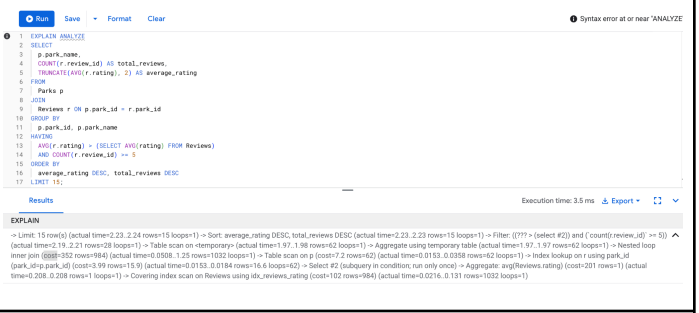
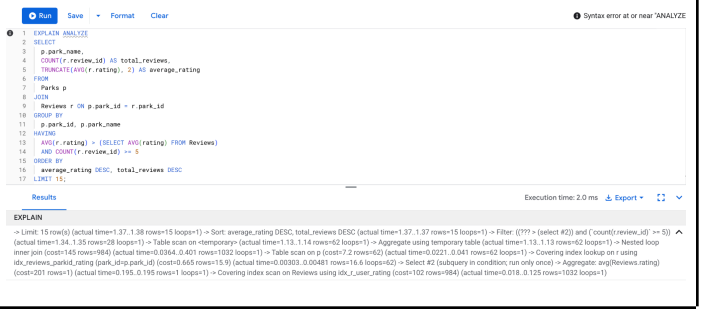
Execution time: 4.0 ms Export

username	park_name	actual_rating	avg_user_rating
rachel36	Everglades	1	3.3333
suttontracy	Yosemite *	1	3.3333
teresajones	park_name	1	3.3333
matthewwright	Petrified Forest	2	4.0000
rglover	Great Sand Dunes	1	3.0000
vasquezjoseph	Mammoth Cave	1	3.0000
eileen55	Everglades	1	3.0000
karenlopez	Dry Tortugas	1	3.0000
marshnathan	White Sands	1	3.0000
debra11	White Sands	1	3.0000
slowery	Great Sand Dunes	1	3.0000
qrasmussen	Petrified Forest	1	3.0000
kristen57	Glacier	1	3.0000
kfaulkner	Grand Canyon *	1	3.0000
xjacobs	Cuyahoga Valley	2	4.0000

Rows per page: 100 1 – 15 of 15

Part 2:

Query 1: Top-Rated Parks by Average Rating and Review Count

Stage	Cost	Screenshot
Initial	368	
CREATE INDEX idx_reviews_rating ON Reviews (rating);	352	
CREATE INDEX idx_parks_name ON Parks (park_name);	352	
CREATE INDEX idx_reviews_parkid_rating ON Reviews (park_id, rating);	145	

Explanation: I selected the composite index from Design 3 as the final design. It produced the lowest query cost 145, representing a 60.6% performance improvement over the baseline 368.

While the other indexes provided minimal to no benefit, this covering index directly addressed the query's main bottlenecks by optimizing both the JOIN condition and the heavy aggregation/ordering operations on the rating column within a single, efficient data structure.

Query 2: Most Active Trip Planners and Reviewers

Stage	Cost	Screenshot
Initial	1178	
CREATE INDEX idx_users_username ON Users(username);	1178	
CREATE INDEX idx_trips_covering ON TripPlans(user_id, trip_plan_id);	1178	

CREATE INDEX idx_reviews_covering ON Reviews(user_id, review_id);	1178	

Explanation: After testing the three proposed indexes, the query cost remained unchanged at 1178, indicating that the indexing did not lead to any noticeable performance improvements. It seems that the database's query planner determined the existing structure was already optimal for the query, as it likely relied on the primary key and other default indexing strategies. The query involves both aggregation and multiple JOINs, and the planner likely opted not to utilize the newly created indexes. As a result, the added indexes did not help in speeding up the core operations, which led to the conclusion that the original index design remains the most efficient for this query.

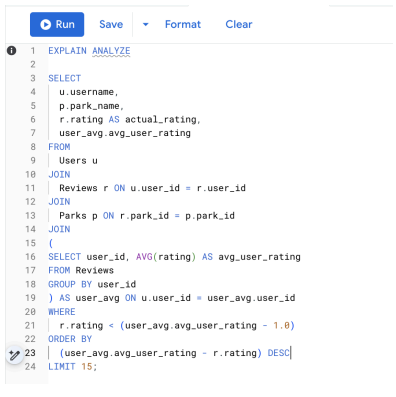
Query 3: Parks with Above-Average User Notes for a Trip Plan

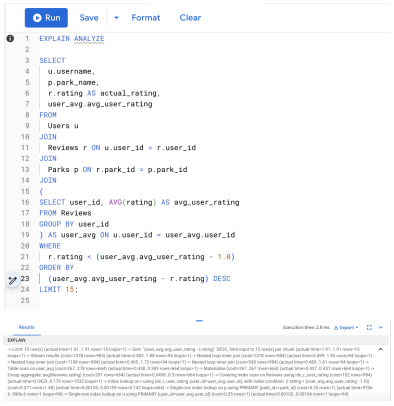
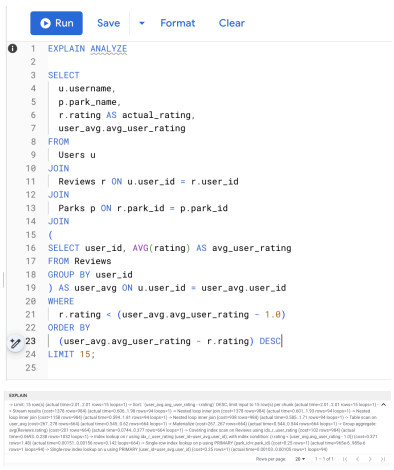
Stage	Cost	Screenshot
Initial	26.8	
CREATE INDEX idx_p_name_state ON Parks (park_name, state);	26.8	

CREATE INDEX idx_tpp_user_notes ON TripPlanParks (user_notes(255));	29	
CREATE INDEX idx_tpp_selective ON TripPlanParks (park_id, user_notes(255));	26.8	

Explanation: The query performance analysis showed that no additional indexes provided a cost improvement for the advanced query. The estimated cost remained consistently at the baseline of 26.8 for almost every change. Specifically, tests targeting grouping/sorting (Test 1: idx_p_name_state on Parks) and selective coverage (Test 4: idx_tpp_selective on TripPlanParks) both matched the baseline cost, resulting in a 0% performance gain. Additionally, the index targeting filtering (Test 2: idx_tpp_user_notes) caused a noticeable increase, raising the cost to 29 (a -8.2% change). Consequently, the optimal index design is to add no new custom indexes, relying solely on the database's existing Primary Key and Foreign Key indexes. This outcome suggests the database is already highly optimized for the current dataset, likely because the existing Foreign Key index on the join column (TripPlanParks.park_id) efficiently handles the most expensive operation, and the additional queries of maintaining new custom indexes are unnecessary.

Query 4: Park Ratings vs. User's Rating Trend

Stage	Cost	Screenshot
Initial	1378	

		
CREATE INDEX idx_reviews_user_park ON Reviews (user_id, park_id);	1378	
CREATE INDEX idx_reviews_user_id ON Reviews (user_id);	1378	
CREATE INDEX idx_reviews_user_rating ON Reviews (user_id, rating);	1378	

Explanation: The indexing tests showed that the estimated query cost stayed exactly the same at 1378 for the starting plan and all three index configurations we tried. This happened because the database's query planner likely decided the default Primary Key structure was already the fastest way to run this complex query, making our added indexes useless. The query is complicated because it calculates an average rating in a subquery and then filters using that average which

means the planner couldn't use simple index lookups. Instead, it processes a large part of the Reviews table's data anyway. Since the PK is so fast for retrieving entire rows, the optimizer determined that the extra work of navigating a secondary index tree (like Reviews(user_id, rating)) wasn’t worth it. Because none of our indexing changes actually lowered the cost, we concluded that the Final Index Design Selection remains the initial Primary Key structure, as the secondary indexes provided no measurable benefit to this specific query's performance.